

অবশ্যই — সহজ বাংলায় বলি।

এই lesson-এ মূলত ৪টা জিনিস বলা হয়েছে:

- **LLM** কী: এটা মূলত next word predict করে, কিন্তু অনেক data দেখে দেখে language, pattern, reasoning-এর মতো capability তৈরি করে।
- সমস্যা কী: LLM অনেক সময় confident হয়ে ভুল উত্তর দেয় — এটাকেই **hallucination** বলে।
- **RAG** কেন দরকার: RAG আগে document/data থেকে relevant info খুঁজে আনে, তারপর সেই তথ্যের উপর ভিত্তি করে answer দেয়। তাই guess কম হয়, answer বেশি reliable হয়।
- এই **course**-এ কী শিখবে: prompt engineering, model selection, embeddings, vector search, RAG system build করা, আর system evaluation।

আরও সহজ করে:

- শুধু LLM = অনেক সময় **smart guess**
- LLM + RAG = **document** দেখে **answer**

এই course-এ তুমি যা build করবে:

- customer service AI assistant
- semantic search system
- NASA mission নিয়ে RAG-based chatbot project

Python part-টা নিয়েও সহজ করে বলি:

- **function** = reusable code block
- **class** = object বানানোর blueprint
- **self.model** = ওই object-এর ভিতরে রাখা model name/value

তোমার দেওয়া example:

```
python
class Chatbot:

    def __init__(self, model_name):

        self.model = model_name

    def get_model(self):

        return self.model
```

এখানে:

- model_name বাইরে থেকে আসে
- self.model object-এর নিজের variable

- `get_model()` call করলে সেই stored value return করবে

যেমন:

```
python  
bot = Chatbot("gpt-4")
```

```
print(bot.get_model())
```

output:

```
python  
gpt-4
```

আর API key কোথায় পাবে:

- উপরের **Cloud Resources** tab-এ
- সেখানে তোমার Vocareum OpenAI key থাকবে
- voc- দিয়ে শুরু হলে অবশ্যই এই base URL ব্যবহার করতে হবে:

```
python  
https://openai.vocareum.com/v1
```

চাইলে আমি এখন এটা তোমার জন্য একদম ৫০–১০০ শব্দের **reflection answer** হিসেবেও বাংলায় লিখে দিতে পারি।

areee ami bollam sudhu The LLM Landscape bole je chapter ta ache...otar purota in details

আচ্ছা বুঝছি 😊 — তুমি শুধু “**The LLM Landscape**” lesson-ta পুরো detail-e চাও।

একদম ওই chapter-টাই sohoj Bangla-তে ভেঙে দিচ্ছি।

এই lesson-এর main idea কী?

এই lesson basically তোমাকে বোঝায়:

- LLM কী
- LLM কী কী পারে
- সব LLM একরকম না
- prompt দিলে output কেন বদলায়
- business-এ plain LLM enough না কেন
- RAG কেন important

মানে, এটা LLM world-এর একটা **big picture overview**।

এই lesson-এর flow roughly এমন:

1. LLM কী
2. LLM capability কী কী
3. model type / tuning
4. prompting techniques
5. real-world use
6. limitation
7. RAG কেন দরকার

LLM কী — এই lesson-এর context-এ

LLM হলো এমন language model যেটা huge amount text দেখে trained।

এটা text understand করে বলে মনে হয়, but ভিতরে ভিতরে এটা mostly **next token prediction** করে।

মানে:

text

"The sky is ..."

এখানে model predict করে next word কী হতে পারে।

এই simple mechanism থেকেই বড় capability আসে।

তাই lesson-টা বোঝাতে চায়:

LLM simple idea থেকে শুরু হলেও **practical use** অনেক বড়।

এই lesson-এ LLM capability গুলো important

১. Generation

মানে নতুন content তৈরি করা।

Examples:

- email লেখা
- blog লেখা
- product description
- Python code লেখা
- chatbot reply

Lesson-এর matching quiz-এ:

- **Generation → Writing functional Python code from a description**

মানে user description দিল, model code generate করল।

২. Summarization

মানে বড় text ছোট করা।

Example:

- long report → short summary
- meeting transcript → key points
- medical notes → condensed version

Quiz-১:

- **Summarization → A doctor reviewing a condensed version of 200 pages of clinical notes**

মানে doctor সব ২০০ page পড়বে না, short summary দেখবে।

৩. Classification

মানে text-কে category-তে ফেলা।

Examples:

- positive / negative review
- spam / not spam
- urgent / non-urgent email
- toxic / safe comment

Quiz-১:

- **Classification → Determining if a customer review is positive or negative**

৪. Reasoning

মানে step-by-step problem solve করা।

Examples:

- logic puzzle
- math problem
- multi-step decision
- comparing options

Quiz-১:

- **Reasoning → Solving multi-step logic problems by thinking step-by-step**

এখানে একটা important point আসে:

কোন technique LLM-এর logic/math performance improve করে?

Answer:

- **Chain-of-Thought prompting**

মানে model-কে বলা:

```
text
```

```
Think step by step
```

এতে model problem-টা ধাপে ধাপে solve করতে চেষ্টা করে।

এই lesson-এ এটা important কারণ সব prompt same result দেয় না।

Chain-of-Thought prompting কেন useful?

কারণ অনেক problem direct answer চাইলে model jump করে ভুল করতে পারে।

কিন্তু step-by-step করতে বললে:

- intermediate steps বের করে
- reasoning clearer হয়
- logic/math task-এ accuracy বাড়ে

Example:

Without CoT:

```
text
```

```
What is  $27 \times 14$ ?
```

With CoT:

```
text
```

```
Solve step by step: what is  $27 \times 14$ ?
```

দ্বিতীয় prompt-এ model usually better কাজ করে।

এই lesson-এ আরেকটা hidden message আছে: **Prompting matters.**

মানে শুধু model powerful হলেই হবে না,

তুমি কীভাবে instruction দিচ্ছে সেটাও important।

LLM-এর different “versions” বা behavior

এই lesson-এ usually বোঝানো হয় যে সব model same না।

একটা raw model আর instruction-following model-এর behavior আলাদা।

সহজ করে:

১. Base / raw model

- text pattern শিখেছে
- next word predict করে
- but user instruction follow করতে খুব polished না

২. Instruction-tuned model

- instruction better follow করে
- যেমন:

text

Summarize this in 3 bullet points

এটা ভালোভাবে মানে

৩. Dialogue/chat tuned model

- conversation-এর জন্য optimized
- follow-up handle করতে পারে
- assistant style answer দেয়

এই distinction important কারণ student যেন বুঝে: সব LLM শুধু “smart brain” না — **training/tuning** অনুযায়ী **behavior** বদলায়।

Lesson-এর business angle

এই lesson শুধু technical capability না, practical use-ও বোঝায়।

LLM এখন অনেক জায়গায় use হয়:

- customer support
- coding assistant
- document search
- report summary
- content writing
- healthcare notes summary
- review analysis

মানে LLM একটা general-purpose language engine হয়ে গেছে।

একটা বড় idea হলো: আগে powerful AI use করা hard ছিল,
এখন API-এর মাধ্যমে developers সহজে use করতে পারে।

মানে:

- নিজে model train করতে হবে না
- API call করে use করা যায়
- তাই অনেক company দ্রুত AI feature build করতে পারে

এই lesson-এ limitation-ও important

LLM powerful হলেও perfect না।

Main limitation:

- hallucination

Hallucination মানে: model confident tone-এ ভুল answer দেয় বা বানিয়ে বলে

Example: তুমি জিজ্ঞেস করলে company policy কী কিন্তু model policy document actually জানে না তবুও একটা polished answer বানিয়ে দিতে পারে

এটাই dangerous।

এই জায়গা থেকেই lesson RAG-এর দিকে যায়।

RAG কী — lesson-এর সবচেয়ে important business takeaway

RAG = Retrieval-Augmented Generation

এটা lesson-এর reflection question-এও আছে, so এটা খুব important।

ভেঙে বলি:

- Retrieval = relevant info খুঁজে আনা
- Augmented = সেই info prompt/context-এ যোগ করা
- Generation = তারপর answer generate করা

মানে:

text

User asks question

→ system searches trusted documents

→ relevant passages finds

→ LLM sees those passages

→ LLM answers using them

কেন দরকার?

কারণ plain LLM:

- সব company-specific info জানে না
- latest info জানে না
- ভুল করতে পারে

RAG:

- trusted source use করে
- answer grounded করে
- hallucination কমায়
- internal docs use করতে দেয়
- serious business use-case possible করে

এই lesson-এর reflection question-এর core idea exactly এটা: **RAG serious business application-এর জন্য necessary কারণ reliability দরকার।**

Business example

ধরো company-র HR bot বানানো হলো।

User asks:

text

How many annual leave days do interns get?

Plain LLM:

- guess করতে পারে
- ভুল policy বলতে পারে

RAG system:

- HR policy document search করবে
- intern leave section বের করবে
- সেই context দিয়ে answer দেবে

এখন answer:

- more accurate
- document-based
- trustworthy

এই lesson-এর big contrast

Lesson-টা basically এই contrast শেখায়:

- **LLM alone = powerful but risky**
- **LLM + grounding/RAG = useful for real business**

এটা খুব important exam/quiz/reflection point।

তুমি যদি পুরো lesson এক paragraph-এ ধরতে চাও:

“The LLM Landscape” lesson explains what LLMs are, what they can do, how prompting and tuning affect their behavior, and why they are useful in real applications. It also highlights that LLMs can hallucinate, which makes them unreliable for serious business use unless they are grounded with trusted information. That is why RAG is important: it retrieves relevant documents first and then uses the LLM to generate a more accurate answer.

এখন quiz part short revision

১. Which technique significantly improves an LLM's ability to solve logic and math problems by instructing it to break down its thinking process?

Correct answer:

- **Chain-of-Thought prompting**

২. Matching:

- **Generation** → Writing functional Python code from a description
- **Summarization** → A doctor reviewing a condensed version of 200 pages of clinical notes
- **Reasoning** → Solving multi-step logic problems by thinking step-by-step
- **Classification** → Determining if a customer review is positive or negative

Reflection question answer idea

Question: What is Retrieval-Augmented Generation (RAG), and why is it necessary for serious business applications of LLMs?

Simple answer: RAG is a method where the system first retrieves relevant information from trusted documents and then gives that information to the LLM to generate an answer. It is necessary for serious business applications because plain LLMs can hallucinate or give incorrect answers. RAG makes answers more accurate, reliable, and grounded in real company data.

Bangla version: RAG হলো এমন একটি পদ্ধতি যেখানে আগে trusted document বা database থেকে relevant information আনা হয়, তারপর সেই information ব্যবহার করে LLM answer তৈরি করে। এটা serious business

application-এর জন্য দরকার, কারণ plain LLM অনেক সময় ভুল বা বানানো উত্তর দিতে পারে। RAG answer-কে বেশি accurate, reliable, এবং real document-based করে।

সবশেষে ultra-short recap:

- LLM = language model
- next word predict করতে করতে useful skill দেখায়
- main capabilities = generation, summarization, classification, reasoning
- reasoning improve করতে = chain-of-thought prompting
- LLM useful but hallucinate করতে পারে
- তাই business-এ RAG দরকার
- RAG = retrieve first, then generate

চাইলে next message-এ আমি এটা আরও **lesson note style**-এ লিখে দিতে পারি —

একদম:

- topic by topic
- important lines
- exam point
- 2 মিনিটে revise করার মতো short note

বা চাইলে **Bangla reflection answer**-টাও direct submission-এর মতো লিখে দিই।

হ্যাঁ, এবার এই “**Demo: LLM Chatbot**” পুরোটা একদম **sohoj vabe, step by step** বুঝাই।

এটা basically শেখাচ্ছে: কীভাবে একটা **chatbot** বানাতে হয়, যাতে সে আগের কথা “মনে আছে” এমন **behave** করে।

সবচেয়ে important line আগে:

LLM-এর নিজের কোনো **memory** নাই।

তবুও chatbot আগের কথা মনে রাখে মনে হয়, কারণ আমরাই প্রতি **API call**-এ আগের সব **message** আবার পাঠাই।

একদম core idea:

```
python
messages = [
    {"role": "user", "content": "Hi"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "Can you explain it again?"}
```

এখানে model “it” বুঝতে পারে, কারণ আগের message-গুলোও তাকে দেওয়া হয়েছে।

এখন পুরো lesson-ta ভেঙে বলি।

প্রথম অংশ: “From Simple Responses to Intelligent Conversations”

এখানে lesson বলছে:

- ChatGPT / Claude / Gemini natural লাগে
- মনে হয় bot আগের কথা remember করছে
- but আসলে model কিছুই remember করে না
- প্রতি **request**-এই **fresh start**

মানে, তুমি যদি শুধু এটা পাঠাও:

```
python
```

```
{"role": "user", "content": "What colors?"}
```

তাহলে model জানবে না “কিসের colors?”

কিন্তু যদি আগের message-ও পাঠাও:

```
python
```

```
[  
    {"role": "user", "content": "Do you have wireless headphones?"},  
    {"role": "assistant", "content": "Yes, we do."},  
    {"role": "user", "content": "What colors?"}  
]
```

তাহলে model বুঝবে “colors of headphones”.

দ্বিতীয় অংশ: Stateless Nature of LLMs

Stateless মানে:

model previous API call-এর কিছু save করে রাখে না।

সহজ example:

- website server তোমাকে page দিল
- তারপর forget করে গেল
- next click-এ আবার info পাঠাতে হয়

LLM-3 same:

- ১ম call-এ question দিলে answer দিল
- ২য় call-এ সে ১ম call মনে রাখে না
- unless তুমি আগের conversation আবার পাঠাও

তাই chatbot বানানোর সময় **conversation history** maintain করতে হয়।

তৃতীয় অংশ: Illusion of Memory

এখানে lesson দেখাচ্ছে memory আসলে fake memory।

Turn 1:

```
python
```

```
messages = [  
    {"role": "user", "content": "What's the weather like today?"}
```

```
]
```

Turn 2:

```
python
```

```
messages = [  
    {"role": "user", "content": "What's the weather like today?"},  
    {"role": "assistant", "content": "I don't have access to real-time weather..."},  
    {"role": "user", "content": "Should I bring an umbrella?"}
```

```
]
```

এখন model “umbrella” কেন বুঝবে?

কারণ:

- weather-এর কথা আগে ছিল
- assistant-এর আগের উত্তরও আছে
- তাই নতুন প্রশ্নটা old context-এর সাথে connect করতে পারে

Quiz answer এখানেই:

Model connect করতে পারে because **the full conversation history is included in the API call.**

চতুর্থ অংশ: Message Roles

প্রতি message-এর একটা role থাকে।

এই role model-কে বলে দেয় কোন message কী কাজ করছে।

৩টা main role:

1. **system**
2. **user**
3. **assistant**

এখন একে একে বুঝি।

১. system role

```
python
{
    "role": "system",
    "content": "You are a helpful tech support assistant. Be patient and clear."
}
```

এটা user দেখে না, but bot-এর behavior control করে।

এটা bot-কে বলে:

- তুমি কে
- কী tone-এ কথা বলবে
- কী help করবে
- কী করবে না

সহজভাবে: **system prompt = bot-এর brain-এর instruction**

যদি system prompt বদলাও:

```
python
"You are a rude assistant."
```

তাহলে answer style বদলে যাবে।

আবার যদি দাও:

```
python
```

```
"You are a polite customer support bot. Ask clarifying questions."
```

তাহলে bot polite হবে, clarification চাইবে।

২. user role

```
python
```

```
{  
  
    "role": "user",  
  
    "content": "My laptop won't turn on"  
  
}
```

এটা হলো মানুষের message।

মানে:

- question
 - request
 - complaint
 - statement
-

৩. assistant role

```
python
```

```
{  
  
    "role": "assistant",  
  
    "content": "Let's troubleshoot this step by step."  
  
}
```

এটা bot-এর আগের answer।

এটাও history-তে রাখতে হয়, কারণ future response-এ model যেন জানে সে আগে কী বলেছিল।

খুব important point:

শুধু user message রাখলে হবে না।

assistant-এর আগের **reply**-ও রাখতে হবে।

না হলে conversation broken হয়ে যাবে।

পঞ্চম অংশ: Building Stateful Conversations

এটাই পুরো lesson-এর heart।

Code:

```
python

conversation = []

def chat(user_message):

    conversation.append({

        "role": "user",

        "content": user_message

    })

    response = client.chat.completions.create(

        model="gpt-3.5-turbo",

        messages=conversation,

        temperature=0.7,

        max_tokens=150

    )

    assistant_message = response.choices[0].message.content

    conversation.append({
```

```
        "role": "assistant",

        "content": assistant_message

    })
```

```
    return assistant_message
```

এখন line by line বুঝি।

```
python
```

```
conversation = []
```

মানে একটা list বানানো হলো, যেখানে পুরো chat history থাকবে।

```
python
```

```
def chat(user_message):
```

মানে chat নামে function বানানো হলো।

এটা user-এর message নেবে।

```
python
```

```
conversation.append({
```

```
    "role": "user",
```

```
    "content": user_message
```

```
})
```

মানে user যা লিখেছে, সেটা history-তে save করা হলো।

```
python
```

```
response = client.chat.completions.create(
```

```
    model="gpt-3.5-turbo",
```

```
    messages=conversation,
```



```
temperature=0.7,  
  
max_tokens=150
```

এখানে API call হচ্ছে।

- model="gpt-3.5-turbo" → কোন model use হবে
- messages=conversation → পুরো history পাঠানো হচ্ছে
- temperature=0.7 → একটু natural / varied response
- max_tokens=150 → answer কত বড় হতে পারবে

সবচেয়ে important: **messages=conversation**

এটাই context maintain করছে।

```
python
```

```
assistant_message = response.choices[0].message.content
```

এখানে API response থেকে bot-এর actual text বের করা হচ্ছে।

সহজভাবে:

- response = পুরো response object
- choices[0] = first answer
- message.content = answer text

```
python
```

```
conversation.append({  
  
    "role": "assistant",  
  
    "content": assistant_message  
  
})
```

এখানে bot-এর answer history-তে save করা হচ্ছে।

এটা না করলে next turn-এ bot জানবে না সে আগে কী বলেছিল।

```
python
```

```
return assistant_message
```

মান্নে function শেষে bot-এর answer return করবে।

ষষ্ঠ অংশ: Following the Conversational Thread

Example:

```
python
```

```
user_msg_1 = "My application keeps crashing"
```

```
bot_response_1 = chat(user_msg_1)
```

এখানে bot crash issue জানল।

তারপর:

```
python
```

```
user_msg_2 = "Version 2.5"
```

```
bot_response_2 = chat(user_msg_2)
```

এখানে user শুধু version বলল।

সে আবার “my application keeps crashing” repeat করল না।

তবুও bot বুঝবে, কারণ আগের message history-তে আছে।

তারপর:

```
python
```

```
user_msg_3 = "When I try to export a file"
```

```
bot_response_3 = chat(user_msg_3)
```

এখানে “it” বা “when I try to export” crash issue-এর সাথে connect হবে।

কারণ model পুরো thread দেখছে।

সপ্তম অংশ: Context Window Limit

এখানে lesson বলছে:

সব history সবসময় পাঠানো যায় না।

কারণ model-এর একটা **token limit** আছে।

যেমন:

- GPT-3.5-turbo → 4096 tokens
- GPT-4 → বেশি হতে পারে

যদি conversation খুব বড় হয়:

- API error দিতে পারে
- সব history fit নাও করতে পারে

তাই long conversation handle করার strategy লাগে।

অষ্টম অংশ: Long conversation manage করার 3টা উপায়

১. Truncation

মানে পুরনো message কেটে ফেলা, recent কিছু রাখা।

```
python
```

```
recent_history = conversation[-max_history:]
```

মানে list-এর শেষের কিছু message রাখা।

Benefit:

- simple
- easy

Problem:

- old context হারিয়ে যায়
-

২. Summarization

পুরনো chat short summary বানিয়ে রাখা।

মানে:

- old 10 exchange নিয়ে summary বানাও

- full old messages বাদ দাও
- summary রেখে দাও

Benefit:

- important info কিছুটা থাকে

Problem:

- extra API call লাগে
 - summary perfect নাও হতে পারে
-

৩. Selective Context

শুধু relevant পুরনো message পাঠানো।

মানে:

- user নতুন question করল
- system old history থেকে relevant messages খুঁজে নিল
- শুধু সেগুলো পাঠাল

Benefit:

- smart
- token save হয়

Problem:

- implementation complex
-

নবম অংশ: Exercise – Customer Service Chatbot

এখন lesson demo থেকে exercise-এ যাচ্ছে।

তোমাকে একটা chatbot বানাতে হবে ShopEasy / customer service style-এ।

Bot handle করবে ৫ ধরনের intent:

- `order_status`
- `product_info`
- `returns`
- `technical_support`
- `general`

মানে user message আগে classify হবে, তারপর response generate হবে।

এখন class structure বুঝি

```
python
```

```
class CustomerServiceBot:
```

মানে একটা class বানানো হচ্ছে।

এই class থেকে bot object তৈরি হবে।

এই class-এর main methods:

- `__init__()`
- `classify_intent()`
- `generate_response()`
- `reset_conversation()`
- `get_conversation_summary()`

এখন একে একে।

Task 1: `__init__()` method

```
python
```

```
def __init__(self, api_key: str, model: str = "gpt-3.5-turbo"):
```

এটা constructor।

Object create করার সাথে সাথে run হয়।

যেমন:

```
python
```

```
bot = CustomerServiceBot(api_key)
```

এখানে `__init__()` automatically run করবে।

Line by line:

```
python
```

```
self.client = OpenAI(api_key=api_key)
```

মানে OpenAI client তৈরি হচ্ছে।

`self.client = এই bot object-এর client`

```
python
```

```
self.model = model
```

মানে model name save হচ্ছে।

```
python
```

```
self.conversation_history = []
```

মানে empty list বানানো হলো, যেখানে chat history থাকবে।

```
python
```

```
system_prompt = self._get_system_prompt()
```

মানে system prompt text আনা হচ্ছে।

```
python
```

```
self.conversation_history.append({
```

```
    "role": "system",
```

```
    "content": system_prompt
```

```
})
```

মানে history-র শুরুতেই system prompt রাখা হচ্ছে।

খুব important: conversation history শুরুতেই system message থাকবে।

Vocareum key routing

যদি key শুরু হয় voc- দিয়ে, তাহলে:

```
python
```

```
self.client = OpenAI(  
    base_url="https://openai.vocareum.com/v1",  
    api_key=api_key  
)
```

আর normal key হলে:

```
python
```

```
self.client = OpenAI(api_key=api_key)
```

এটা খুব important, না হলে 401 error আসতে পারে।

Task 2: System Prompt design

System prompt define করবে bot-এর:

- role
- tone
- capability
- boundary

Example idea:

```
python
```

```
You are a helpful customer service assistant for ShopEasy.
```

```
Be polite, clear, and patient.
```

```
Help with order status, product information, returns, and technical support.
```

```
If you do not know something, say so clearly and offer next steps.
```

এখানে lesson তোমাকে prompt engineering practice করছে।

Task 3: classify_intent()

এই method user message দেখে category decide করবে।

যেমন:

- "Where is my order?" → order_status
- "Do you have laptops?" → product_info
- "What's your return policy?" → returns
- "I can't log in" → technical_support
- "Hello" → general

এখানে usually LLM-কে prompt দিয়ে বলা হয়: "Only return one of these labels..."

Important points:

- classification-এর জন্য temperature=0
- output validate করতে হবে

কারণ model unexpected কিছু লিখে দিতে পারে।

Example logic:

```
python
```

```
intent = response.choices[0].message.content.strip().lower()
```

```
valid_intents = ['order_status', 'product_info', 'returns', 'technical_support',  
'general']
```

```
if intent not in valid_intents:
```

```
    intent = 'general'
```

এখানে:

- strip() → extra space remove
- lower() → small letter
- valid না হলে fallback = general

Task 4: generate_response()

এটাই main chatbot function।

Code flow:

```
python
```

```
if intent is None:
```



```
intent = self.classify_intent(user_message)
```

মান্নে intent আগে থেকে না দিলে, first classify করবে।

```
python
```

```
self.conversation_history.append({  
    "role": "user",  
    "content": user_message  
})
```

user message history-তে add করবে।

```
python
```

```
response = self.client.chat.completions.create(  
    model=self.model,  
    messages=self.conversation_history,  
    temperature=0.7,  
    max_tokens=300  
)
```

এখানে full history দিয়ে response generate হচ্ছে।

```
python
```

```
assistant_message = response.choices[0].message.content
```

bot-এর answer বের করা হচ্ছে।

```
python
```

```
self.conversation_history.append({
```

```
        "role": "assistant",

        "content": assistant_message

    ))
```

bot-এর answer history-তে save হচ্ছে।

```
python

return assistant_message
```

answer return হচ্ছে।

try/except কেন?

```
python

try:

    ...

except Exception as e:
```

কারণ API call fail করতে পারে:

- wrong key
- network issue
- token limit
- server issue

তাই crash না করে friendly message return করবে।

Task 5: reset_conversation()

```
python

self.conversation_history = [{

    "role": "system",

    "content": self._get_system_prompt()

}]
```

মানে পুরো পুরনো chat delete করে শুধু system prompt রাখা।

কেন দরকার?

- new customer
- start over
- conversation off-track

Reset-এর পর history length হবে 1।

Task 6: `get_conversation_summary()`

এটা human handoff-এর জন্য summary বানাবে।

যদি history-তে system prompt ছাড়া কিছু না থাকে:

```
python
```

```
if len(self.conversation_history) <= 1:
```

```
    return "No conversation to summarize yet."
```

মানে summarize করার মতো chat হয়নি।

তারপর:

```
python
```

```
summary_messages = self.conversation_history + [{
```

```
    "role": "user",
```

```
    "content": summary_prompt
```

```
}]
```

মানে existing conversation-এর শেষে summary request add করা হচ্ছে।

তারপর API call:

```
python
```

```
response = self.client.chat.completions.create(
```

```
    model=self.model,
```

```
    messages=summary_messages,
```

```
temperature=0.3,  
max_tokens=200
```

এখানে temperature=0.3 কারণ summary factual হওয়া ভালো, বেশি creative না।

Task 7: main() function

এটা program run করার main entry point।

```
python
```

```
api_key = os.getenv("OPENAI_API_KEY")
```

মান্নে environment variable থেকে API key নিচ্ছে।

```
python
```

```
if not api_key:
```

```
    print("Error: Please set OPENAI_API_KEY environment variable")
```

```
    return
```

key না থাকলে stop করবে।

```
python
```

```
bot = CustomerServiceBot(api_key)
```

bot object create করছে।

তারপর chat loop:

```
python
```

```
while True:
```

মান্নে infinite loop, যতক্ষণ user quit না করে।

```
python
```

```
user_input = input("You: ").strip()
```

user input নিচ্ছে।

```
python
```

```
if not user_input:
```

```
    continue
```

empty input হলে skip করবে।

Commands:

```
python
```

```
if user_input.lower() in ['quit', 'exit']:
```

program বন্ধ করবে।

```
python
```

```
if user_input.lower() == 'reset':
```

conversation reset করবে।

```
python
```

```
if user_input.lower() == 'summary':
```

conversation summary দেখাবে।

Otherwise:

```
python
```

```
response = bot.generate_response(user_input)
```

```
print(f"\nBot: {response}\n")
```

মানে normal chatbot response generate করবে।

Running the chatbot

```
bash
```

```
export OPENAI_API_KEY="your-api-key-here"
```

```
python exercises/starter/customer_service_bot.py
```

এখানে:

- first line key set করছে
 - second line file run করছে
-

Sample interaction-এর meaning

User:

```
text
```

```
Hi, I need help with my order
```

Bot order-related context ধরল।

User:

```
text
```

```
It's order #12345. I placed it 3 days ago.
```

এখন bot order number + time info পেল।

User:

```
text
```

```
Where is it now?
```

এখানে “it” মানে order/package।

Bot বুঝতে পারছে কারণ history আছে।

তারপর summary দিলে পুরো conversation short করে দেয়।

শেষের “Chatbot Issues and Testing” section

এটা খুব important troubleshooting part।

১. Forgetting to add assistant message

Wrong:

```
python
self.conversation_history.append({"role": "user", "content": user_message})

response = ...

return response.choices[0].message.content
```

Problem:

- bot response history-তে save হয়নি
- next turn-এ bot জানবে না সে কী বলেছিল

Correct:

```
python
assistant_message = response.choices[0].message.content

self.conversation_history.append({"role": "assistant", "content": assistant_message})

return assistant_message
```

২. System prompt missing

Wrong:

```
python
self.conversation_history = []
```

Problem:

- bot-এর behavior instruction নেই

Correct:

```
python
```

```
self.conversation_history = [{  
    "role": "system",  
    "content": self._get_system_prompt()  
}]
```

৩. Wrong temperature for classification

Wrong:

```
python
```

```
temperature=0.7
```

Problem:

- classification random হতে পারে

Correct:

```
python
```

```
temperature=0
```

কারণ classification-এ stable answer দরকার।

8. Not validating intent

Wrong:

```
python
```

```
intent = response.choices[0].message.content.strip()
```

```
return intent
```

Problem:

- model হয়তো order status লিখবে, বা sentence লিখবে

Correct: valid list check করতে হবে।

৫. Missing error handling

Wrong: API fail করলে পুরো program crash করবে।

Correct: try/except use করতে হবে।

৬. Long conversation token limit

যদি অনেক বড় history হয়, truncate করতে পারো:

```
python
def truncate_history(self, max_messages=20):
    if len(self.conversation_history) > max_messages:
        system_prompt = self.conversation_history[0]
        recent_messages = self.conversation_history[-(max_messages-1):]
        self.conversation_history = [system_prompt] + recent_messages
```

এখানে:

- system prompt always keep
 - recent messages keep
 - old messages drop
-

Testing file test_chatbot.py

এটা check করবে ৪টা জিনিস:

1. intent classification ঠিক আছে?
2. context-aware response দিচ্ছে?
3. summary কাজ করছে?
4. reset কাজ করছে?

Example:

```
python
actual = bot.classify_intent(message)
```

এখানে actual intent বের হচ্ছে।

```
python
```

```
status = "✓" if actual == expected else "✗"
```

মান্নে expected-এর সাথে compare করছে।

Context test:

```
python
```

```
bot.generate_response("Do you have wireless headphones?")
```

```
response = bot.generate_response("What colors?")
```

এখানে second response ideally headphones-এর colors নিয়ে হবে।

Summary test:

```
python
```

```
bot.generate_response("I need to return a laptop")
```

```
bot.generate_response("I bought it 2 weeks ago")
```

```
bot.generate_response("It doesn't turn on")
```

```
summary = bot.get_conversation_summary()
```

এখানে summary-তে return issue + timeline + problem mention থাকা উচিত।

Reset test:

```
python
```

```
before = len(bot.conversation_history)
```

```
bot.reset_conversation()
```

```
after = len(bot.conversation_history)
```

reset-এর পর after == 1 হওয়া উচিত, কারণ শুধু system prompt থাকবে।

এখন পুরো lesson-এর main idea ৮ লাইনে:

- LLM stateless
- memory illusion আসে full conversation history পার্থানোর মাধ্যমে
- messages list-এ system, user, assistant roles থাকে
- system prompt bot-এর behavior control করে
- user message add করতে হয়
- assistant response-ও history-তে add করতে হয়
- long conversation token limit hit করতে পারে
- chatbot build করতে classify + respond + reset + summary useful

সবচেয়ে exam/useful points:

- **LLM remembers because we resend history**
- **system prompt defines behavior**
- **assistant messages must also be stored**
- **classification → temperature 0**
- **Vocareum key হলে base_url লাগবে**
- **long chats need truncation/summarization/selective context**

চাইলে next message-এ আমি এটা আরও easy করে **code by code line-by-line** বুঝাই, যেমন:

- `__init__()` line by line
- `classify_intent()` line by line
- `generate_response()` line by line
- `main()` line by line

অথবা চাইলে আমি তোমার জন্য এই পুরো **lesson-এর short exam note** বানিয়ে দিই।

আপনি যে লেখাটি দিয়েছেন, সেটি মূলত "প্রম্পট ইঞ্জিনিয়ারিং" (Prompt Engineering) এবং লার্জ ল্যাঙ্গুয়েজ মডেল (LLM - যেমন চ্যাটজিপিটি বা জেমিনি) কীভাবে কাজ করে, তার একটি দারুণ ও বিস্তারিত গাইড।

আমি পুরো বিষয়টি খুব সহজ ভাষায় এবং ধাপে ধাপে বাংলায় বুঝিয়ে দিচ্ছি:

সহজ কথায়, প্রম্পট হলো মানুষের সাথে এআই-এর যোগাযোগের মাধ্যম। আপনি কাউকে কোনো কাজের দায়িত্ব দেওয়ার সময় যেমন বিস্তারিত বুঝিয়ে বলেন—কী করতে হবে, কীভাবে করতে হবে—এআই-এর ক্ষেত্রেও ঠিক তাই। আপনি লিখে যা নির্দেশ দেন, সেটিই হলো প্রম্পট।

অবশ্যই — এটা আসলে বলছে কীভাবে ভালো **prompt** লিখলে **LLM** ভালো **answer** দেয়।

সহজ ভাষায় মূল idea:

- **Basic prompt** = ছোট, vague, context কম
- **Advanced prompt** = role + context + exact task + output format
- যত clear prompt, তত useful answer

উদাহরণ: এটা weak prompt:

```
python
prompt = f"What is {product_name}?"
```

এটা better prompt:

```
python
prompt = f"""You are examining the product documentation.

Based on this info:

{retrieved_docs}

Explain {product_name}, including:

1. features

2. accessories

3. common questions

4. warranty

Use simple language."""
```

কেন better?

- model-কে role দেওয়া হয়েছে
- relevant context দেওয়া হয়েছে
- কী কী cover করতে হবে বলা হয়েছে
- কেমন language use করবে সেটাও বলা হয়েছে

এই lesson-এর ৪টা common problem খুব important:

- **Ambiguity Trap**
prompt অস্পষ্ট হলে answer-ও অস্পষ্ট হবে
যেমন:
 - text

- Tell me about the delivery issue
এর বদলে:
- text
- Explain the delayed shipment for order #12345 that was supposed to arrive on Dec 15, 2024
- **Context Overflow**
বেশি অপ্রয়োজনীয় text দিলে model confused হতে পারে
তাই পুরো document না দিয়ে relevant অংশ দাও
- **Assumption Gap**
model সব জানে ধরে নিলে ভুল হবে
recent/specific info prompt-এ দাও
- **Format Confusion**
output format না বললে model নিজের মতো answer দেবে
তাই clearly বলো:
- text
- Return JSON with fields: summary, technical_details, recommendations

তারপর advanced techniques:

- **Few-shot learning**
শুধু instruction না দিয়ে example দেখাও
মানে:
- text
- এই input হলে এই output
এতে model pattern শিখে better output দেয়
- **Self-consistency**
একই problem কয়েকবার solve করাও, তারপর common answer দেখো
useful যখন reasoning task থাকে
- **Constitutional AI principles**
prompt-এর মধ্যেই safety rules দাও
যেমন:
 - unsure হলে বলবে unsure
 - source ছাড়া কিছু বানাবে না
 - fact আর speculation আলাদা করবে

সব মিলিয়ে lesson-এর main message:

- ভালো prompt মানে শুধু question না
- ভালো prompt = **clear task + context + constraints + format + examples**

এক লাইনে:

- **Garbage prompt দিলে garbage output**
- **Clear prompt দিলে structured, accurate output**

এআই আসলে মানুষের মতো চিন্তা করে না। এটি একটি নির্দিষ্ট বাক্যের পর সবচেয়ে সম্ভাব্য পরবর্তী শব্দটি কী হতে পারে, তা অনুমান করে। ধরুন আপনি লিখলেন, "আকাশের রং..."। এআই তার ডেটাবেস ঘেঁটে দেখবে এই বাক্যের পর "নীল" শব্দটি আসার সম্ভাবনা সবচেয়ে বেশি (যেমন: ৯০%)। এরপর সে "নীল" শব্দটি বসিয়ে দেবে। এই পদ্ধতিটিকেই বলে "নেক্সট টোকেন প্রেডিকশন" বা পরবর্তী শব্দ অনুমান করা।

মডেলের ভেতরে অনেক তথ্য সেভ করা থাকে (যাকে ওয়েটস বা Weights বলে)। কিন্তু আপনি যদি প্রম্পটে পরিষ্কারভাবে আপনার দরকারি তথ্য না দেন, তখন এআই বাধ্য হয়ে তার নিজস্ব ডেটা থেকে কিছু একটা বানিয়ে বলার চেষ্টা করে। যেমন, আপনি যদি শুধু বলেন "ফেরত নীতি বা রিফান্ড পলিসি কী?"—এআই তখন আন্দাজে কোনো একটা পলিসি বলে দিতে পারে। কিন্তু আপনি যদি বলেন, "ইলেকট্রনিক্স পণ্য ৩০ দিনের মধ্যে নষ্ট হলে রিফান্ড পলিসি কী?"—তখন এআই সঠিক ও সুনির্দিষ্ট উত্তর দিতে পারবে।

একটি নিখুঁত প্রম্পট লিখতে গেলে কয়েকটি বিষয়ের দিকে খেয়াল রাখতে হয়:

- **সিস্টেম প্রম্পট (System Prompt):** এআই-কে তার ভূমিকা ঠিক করে দেওয়া। (যেমন: "তুমি একজন দক্ষ কাস্টমার সাপোর্ট এজেন্ট...")।
- **কথোপকথনের ইতিহাস (Chat History):** আগের কথার সূত্র ধরে রাখা, যাতে এআই প্রসঙ্গের বাইরে না যায়।
- **ইউজার রিকোয়েস্ট (User Request):** আপনি ঠিক কী চাইছেন, তা একদম পরিষ্কার করে বলা।
- **চেইন অফ থট (Chain of Thought):** এআই-কে ধাপে ধাপে চিন্তা করতে বলা (Think step-by-step)। এতে এআই-এর ভুল করার সম্ভাবনা অনেক কমে যায়।

একই ধরনের প্রম্পট বারবার না লিখে একটি ছাঁচ বা ফ্রেম তৈরি করে রাখাকে প্রম্পট টেমপ্লেট বলে। যেমন:
"*{product category}* পণ্যটি *{timeframe}* দিনের মধ্যে ফেরত দেওয়ার নিয়ম কী?" এখানে আপনি চাইলে শুধু ব্র্যাকেটের ভেতরের শব্দগুলো বদলে ল্যাপটপ, মোবাইল বা পোশাক বসিয়ে দিয়ে বারবার ব্যবহার করতে পারবেন।

আপনি যখন এআই-কে কোনো নির্দেশ দেন, তখন পেছনের কিছু সেটিংস পরিবর্তন করে আপনি উত্তরের ধরন বদলাতে পারেন:

- **টেম্পারেচার (Temperature):** এটি এআই-এর সৃজনশীলতা নিয়ন্ত্রণ করে।
 - **টেম্পারেচার ০ (Low):** উত্তর হবে একদম সোজা, বাস্তবসম্মত এবং টু-দ্য-পয়েন্ট। (যেমন: কোডিং বা তথ্য খোঁজার জন্য)।
 - **টেম্পারেচার ০.৭ - ১.০ (Balanced):** উত্তর হবে কিছুটা সৃজনশীল কিন্তু প্রাসঙ্গিক। (যেমন: গল্প বা আর্টিকেল লেখার জন্য)।
 - **টেম্পারেচার ১.০+ (High):** উত্তর হবে অনেক বেশি এলোমেলো এবং অদ্ভুত।
- **টপ পি (Top P):** এটিও টেম্পারেচারের মতোই কাজ করে। এটি কম সম্ভাবনাময় বা অপ্রাসঙ্গিক শব্দগুলোকে একেবারে ছাঁটাই করে দেয়।
- **ম্যাক্স টোকেন (Max Tokens):** উত্তরের সাইজ কত বড় হবে তা নির্দিষ্ট করে দেওয়া। এটি খুব কম দিলে এআই বাক্যের মাঝখানেই থেমে যেতে পারে।
- **স্টপ সিকোয়েন্স (Stop Sequences):** এআই-কে ঠিক কোথায় গিয়ে থামতে হবে তার নির্দেশ দেওয়া।

আপনার লেখার শেষে যে কুইজগুলো ছিল, সেগুলোর মূল কনসেপ্ট বা উত্তরগুলো নিচে দেওয়া হলো:

1. এলোমেলো উত্তর দেওয়া এবং মাঝপথে থেমে যাওয়া ঠেকাতে: টেম্পারেচার (Temperature) কমিয়ে দিতে হবে (যাতে উত্তর সঠিক হয়) এবং ম্যাক্স টোকেন (Max Tokens) বাড়িয়ে দিতে হবে (যাতে মাঝপথে লেখা থেমে না যায়)।
2. কয়েকটি উদাহরণ দিয়ে এআই-কে বোঝানো: এই পদ্ধতিকে বলা হয় "Few-Shot Learning"। অর্থাৎ, এআই-কে মূল কাজ দেওয়ার আগে কিছু উদাহরণ (Example) দিয়ে প্যাটার্ন বুঝিয়ে দেওয়া।
3. এআই-এর হ্যালুসিনেশন বা বানিয়ে মিথ্যা বলার কারণ: প্রম্পটে পর্যাপ্ত তথ্য বা প্রসঙ্গের (Context) অভাব।
4. **JSON** বা নির্দিষ্ট ফরম্যাটে আউটপুট পাওয়া: এর জন্য প্রম্পটের ভেতরে আউটপুটের ফরম্যাট (Format Confusion দূর করতে) একদম পরিষ্কার করে উল্লেখ করে দিতে হয়।

সংক্ষেপে বলতে গেলে, এআই হলো একটি দারুণ ইঞ্জিন, আর প্রম্পট হলো সেই ইঞ্জিনের স্টিয়ারিং হইল। আপনি স্টিয়ারিং যত ভালোভাবে ঘোরাবেন, এআই আপনাকে তত নিখুঁত গন্তব্যে পৌঁছে দেবে।

ধরো LLM একটা sentence লিখছে। সে একবারে পুরো sentence লেখে না। সে এভাবে কাজ করে:

- প্রথমে ১ম শব্দ বেছে নেয়
- তারপর ২য় শব্দ
- তারপর ৩য় শব্দ
- এভাবে একটার পর একটা

মানে, এটা একটু mobile keyboard suggestion-এর মতো — পরের শব্দ কী হতে পারে সেটা আন্দাজ করে।

উদাহরণ: Prompt:

```
text
```

```
What is an embedding?
```

মডেল ভাবতে পারে প্রথম শব্দ হিসেবে:

- An
- A
- In
- The

এগুলোর মধ্যে যেটার chance সবচেয়ে বেশি, সেটা নেয়। ধরো An নিল।

তারপর আবার ভাববে:

```
text
```

```
An ...
```

এখন পরের শব্দ কী হতে পারে?

- embedding
- embed
- example
- algorithm

আবার সবচেয়ে likely টা নেবে।

logprobs জিনিসটা কী?

- এটা দেখায় মডেল কোন শব্দটা কত confidence নিয়ে বেছে নিয়েছে
- সহজভাবে: মডেল কতটা **sure** ছিল

সংখ্যা কীভাবে বুঝবে:

- -1 → অনেক confident
- -3 → মোটামুটি confident
- -10 → কম confident
- -19 → খুবই কম confident

মনে রাখো:

- কম **negative** = বেশি **confidence**
- বেশি **negative** = কম **confidence**

উদাহরণ: যদি top choices হয়:

```
text
```

An -> -1.2

A -> -2.5

The -> -4.0

```
Sure -> -19
```

এর মানে:

- মডেল An-কে সবচেয়ে বেশি likely ভাবছে
- Sure প্রায় impossible মনে করছে

top_logprobs=5 মানে:

- প্রতিবার next token choose করার সময় top 5 option দেখাবে

এটা কেন দরকার?

- মডেল কোথায় confident আর কোথায় confused, সেটা বোঝা যায়

- weird answer দিলে বুঝতে পারো সে guess করেছে কিনা

এক লাইনে পুরো lesson:

- LLM একটার পর একটা **token predict** করে
- **logprobs** দিয়ে দেখা যায় কোন **token** কত **confidence** নিয়ে **choose** হয়েছে

চাইলে আমি এখন এটা একটা **real mini example** দিয়ে দেখাই, step by step।

হ্যাঁ — এটা বড় লাগছে, কিন্তু আসল জিনিস খুব কম। একদম short and clear করে বলি।

LLM কী করে?

- পুরো sentence একবারে লেখে না
- একটা **token** করে next token predict করে
- প্রতিবার অনেক possible token-এর মধ্যে একটা choose করে

এই choose করার rules-গুলোকেই বলে **inference / decoding settings**

সবচেয়ে important settings:

- **Temperature**
 - randomness control করে
 - low temperature → safe, predictable answer
 - high temperature → creative, risky answer
 - temperature = 0 হলে সাধারণত **greedy decoding**
 - greedy decoding মানে: সবসময় সবচেয়ে probable token নেয়
- **Top P**
 - সব token consider না করে, শুধু এমন কিছু top token রাখে যাদের total probability P পর্যন্ত যায়
 - তারপর ওই ছোট group থেকে sample করে
 - low top_p → কম risky, কম creative
 - high top_p → বেশি variety
- **Max tokens**
 - model কত বড় answer দিতে পারবে তার limit
 - limit hit করলে answer মাঝপথে থেমে যেতে পারে
- **Penalty settings**
 - repeated word/phrase কমাতে use হয়
 - যেমন একই কথা বারবার বলা আটকাতে

সহজ example: Prompt:

text

The weather today is

Model ভাবতে পারে:

```
text
sunny    0.40
warm     0.25
rainy    0.15
great    0.10
banana   0.001
```

এখন:

- greedy decoding হলে sunny নেবে
- high temperature হলে warm বা great-ও আসতে পারে
- top_p ছোট হলে banana-র মতো weird token বাদ পড়ে যাবে

একটা important point:

- temperature=0 হলেও output **100% deterministic** না
- hardware/GPU computation-এর কারণে মাঝে মাঝে slight difference হতে পারে
- তবে mostly same থাকে

Quiz-এর answer: 1. **When greedy decoding for the next token...** সঠিকটা:

```
text
the LLM will not produce a deterministic and reproducible outcome.
```

কারণ lesson-এ বলেছে greedy decoding-ও perfectly deterministic না

2.

When sampling from the next token distribution... সঠিকটা:

```
text
Reducing top p reduces the likelihood of less probable tokens, resulting in less
creative responses.
```

সবচেয়ে important takeaway:

- **temperature** = randomness বাড়ায়/কমায়
- **top_p** = low-probability token cut করে
- **greedy decoding** = সবচেয়ে likely token নেয়

- **max_tokens** = answer length limit

চাইলে আমি এটা এখন **Bangla table** বানিয়ে দিতে পারি: temperature vs top_p vs greedy decoding — এক নজরে বুঝার জন্য।

এখানে lessonটা basically বলছে: LLM-এর **output** শুধু **prompt**-এর উপর না, **settings**-এর উপরও **depend** করে।

সহজ করে এক একটার মানে:

- **Temperature**
 - model কতটা safe vs random হবে
 - 0.1–0.7 → conservative, same ধরনের answer বেশি
 - 0.7–1.0 → balanced, creative but still sensible
 - >1.0 → খুব random, weird answerও আসতে পারে
- **Max tokens**
 - answer কত বড় হতে পারবে তার **hard limit**
 - কম দিলে answer মাঝপথে কেটে যাবে
 - বেশি দিলে token budget waste হতে পারে
- **Context window**
 - model একসাথে মোট যত token handle করতে পারে
 - rule:

```
text
```

```
input tokens + max tokens <= context window
```

- limit cross করলে error হবে বা prompt ছোট করতে হবে
- **Stop sequences**
 - কোন text দেখলে generation থামবে
 - যেমন:

```
python
```

```
stop=["\n"]
```

- মানে newline পেলেই stop

এক লাইনে summary:

- **temperature** = randomness control
- **max_tokens** = answer length limit
- **context window** = total memory limit
- **stop sequence** = কোথায় থামবে

একটা real-life analogy:

- **temperature** = ছাত্র কতটা creative লিখবে

- **max_tokens** = কত লাইনের মধ্যে লিখতে হবে
- **context window** = exam sheet-এ মোট কত জায়গা আছে
- **stop sequence** = full stop/line break এ থামার rule

চাইলে আমি এটা এখন আরও easy করে **Bangla notes + exam-style MCQ answer** বানিয়ে দিতে পারি।

এখানে exercise-টা তোমাকে **LLM inference parameters** হাতে-কলমে **test** করতে বলছে — মানে same model, same prompt, কিন্তু settings বদলালে output কেমন বদলায় সেটা দেখবে।

কি কি করতে বলছে, detail-এ:

- `__init__()`
 - OpenAI client বানাবে
 - model name save করবে

```
python
```

```
self.client = OpenAI(api_key=api_key)
```

```
self.model = model
```

- `generate_with_temperature()`
 - same prompt different temperature দিয়ে run করবে
 - বুঝবে low temp = stable, high temp = creative/random
- `compare_temperatures()`
 - অনেকগুলো temperature loop করে compare করবে

```
python
```

```
for temp in temperatures:
```

```
    results[temp] = self.generate_with_temperature(prompt, temp)
```

- `generate_with_top_p()`
 - top_p change করে nucleus sampling test করবে
 - low top_p = safer/focused
 - high top_p = more diverse
- `generate_with_max_tokens()`
 - response length control করবে
 - ছোট max_tokens দিলে answer কেটে যেতে পারে
- `generate_with_frequency_penalty()`
 - repeated words/phrases কমে কি না দেখবে
 - বেশি penalty দিলে repetition কমবে
- `analyze_logprobs()`
 - model কোন token কত confidence দিয়ে choose করেছে সেটা inspect করবে
 - logprobs=True, top_logprobs=5

তারপর experiments-এ basically এই functions call করে output print করবে:

- Experiment 1 → temperature compare
- Experiment 2 → top_p compare
- Experiment 3 → max_tokens compare
- Experiment 4 → frequency_penalty compare
- Experiment 5 → logprobs inspect

সবচেয়ে important learning:

- **temperature** = randomness
- **top_p** = কতদূর পর্যন্ত candidate token consider করবে
- **max_tokens** = answer length cap
- **frequency_penalty** = repetition control
- **logprobs** = model confidence check

এক লাইনে exercise-এর goal: “**same prompt + different inference settings = different style, length, diversity, confidence**” — এটা practically দেখানো।

চাইলে আমি এখন তোমার জন্য এটা **Bangla line-by-line code walkthrough** করে দিতে পারি, যেন file-এ কোন function-এ কী লিখতে হবে একদম clear হয়।

অবশ্যই — এটা পুরোটা আসলে **prompt engineering**-এর 4টা **main technique** শিখাচ্ছে, আর দেখাচ্ছে কখন কোনটা use করতে হবে।

সবচেয়ে short summary:

- **Zero-shot** = শুধু task দাও
- **Few-shot** = task + 2/3টা example দাও
- **Chain-of-Thought** = task + “step by step” reasoning চাই
- **Structured output** = output কোন format-এ চাই সেটা clearly বলো

এখন একদম detail-এ যাই।

এই demo-র প্রথম অংশ: **Chain-of-Thought (CoT) Prompting**

ওরা প্রথমে same question 2ভাবে করেছে:

1. direct prompt

text

How would you measure the diameter of planet Earth using Eratosthenes' method?

2. CoT prompt

text

How would you measure the diameter of planet Earth using Eratosthenes' method?

Give a step-by-step process to achieve this.

এখানে difference কী?

- direct prompt দিলে model answer দেয়, কিন্তু অনেক সময়:
 - short হয়
 - mixed information দেয়
 - history + explanation + answer একসাথে দেয়
 - exact process clear নাও হতে পারে
- CoT prompt দিলে model:
 - problem-টা ভেঙে step বানায়
 - reasoning দেখায়
 - process clearer হয়
 - ভুল ধরাও easier হয়

মানে CoT হলো: “**final answer** দেওয়ার আগে **intermediate steps** দেখাও”

এখন Eratosthenes method example-টা সহজ করে বুঝি

idea ছিল:

- দুই জায়গায় সূর্যের shadow compare করা
- shadow angle difference থেকে Earth-এর curvature estimate করা
- distance between cities জানা থাকলে Earth circumference বের করা
- circumference থেকে diameter বের করা

CoT style answer সাধারণত এমন হবে:

1. একই দিনে, একই সময়ে দুই জায়গায় shadow measure করো
2. angle difference বের করো
3. দুই জায়গার distance জানো
4. angle/distance ratio থেকে Earth circumference estimate করো
5. তারপর:

text

diameter = circumference / pi

এখানে CoT useful কারণ এটা **multi-step reasoning problem**।

এখন lesson-এর main theory part:

1. Zero-Shot Prompting

Definition: কোন **example** না দিয়ে শুধু **task** বলা।

Example:

text

Summarize this article.

বা

text

Translate this to French.

এটা কখন use করবে:

- task simple হলে
- common task হলে
- model আগে থেকেই task-type জানে
- quick test/prototype করতে চাইলে

ভালো use case:

- summarization
- translation
- definition
- simple sentiment classification

খারাপ use case:

- custom labels
- strict format
- domain-specific task

কেন? কারণ example না দিলে model নিজের মতো format choose করতে পারে।

ধরো prompt:

text

Classify sentiment: "This is great!"

model output দিতে পারে:

- Positive
- The sentiment is positive.
- This expresses positive sentiment.
- Positive sentiment

মান্নে answer correct হলেও **format inconsistent** হতে পারে।

2. Few-Shot Prompting

Definition: **Task**-এর আগে 2–5টা **example** দিয়ে **pattern** শেখানো।

Example structure:

```
python
```

```
prompt = """
```

Example 1:

Input: I love this product

Output: Positive

Example 2:

Input: This is terrible

Output: Negative

Now complete this task:

Input: The service was okay

Output:

```
"""
```

এখানে model বুঝে যায়:

- output short হবে
- label style follow করতে হবে
- explanation দিতে হবে না

Few-shot use করবে যখন:

- consistent format দরকার
- custom categories আছে
- domain-specific language আছে
- specific writing style দরকার

Best number of examples:

- 1 = helpful but weak
- 2–3 = sweet spot
- 4–5 = okay but diminishing returns
- 10+ = usually waste of tokens

কেন few-shot powerful? কারণ model-কে শুধু task না, **pattern**-ও দেখানো হয়।

3. Chain-of-Thought (CoT) Prompting

Definition: **Model-কে reasoning steps** দেখাতে বলা।

Typical trigger phrases:

- Let's think step by step
- Give a step-by-step process
- Show your reasoning
- Explain your thought process
- Break this down into steps

এটা use করবে যখন:

- math
- logic
- multi-step reasoning
- debugging
- decision making
- teaching/explaining

Use করবে না যখন:

- simple fact
 - quick lookup
 - creative writing
 - very short direct answer enough
-

Cupcake example-টা খুব important

Problem:

text

A bakery sells cupcakes for \$3 each.

They offer a 20% discount on orders of 10 or more.

If a customer orders 15 cupcakes, what is the total cost?

Without CoT:

text

\$36

With CoT:

text

Step 1: Base price = $15 \times 3 = 45$

Step 2: Discount applies

Step 3: Discount = $45 \times 0.20 = 9$

Step 4: Final price = $45 - 9 = 36$

Answer: \$36

এখানে interesting point:

- শুধু final answer দেখলে trust কম
- step দেখলে verify করা যায়
- কোথায় ভুল হলো বোঝা যায়

তাই CoT শুধু accuracy না, **trust**-ও বাড়ায়।

4. Structured Output Prompting

Definition: **Output** কোন **format**-এ চাই সেটা **explicitly** বলা।

যেমন:

- JSON

- CSV
- Markdown table
- YAML

কেন দরকার? কারণ free-form text code-এ parse করা hard।

Example without structure:

text

John Smith is 35 years old and lives in Seattle. He works as an engineer at TechCorp.

এটা parse করতে coding extra লাগবে।

Structured prompt:

text

Extract person info in JSON format:

John Smith, 35, lives in Seattle, works as engineer at TechCorp

Expected output:

json

```
{
  "name": "John Smith",
  "age": 35,
  "city": "Seattle",
  "occupation": "engineer",
  "company": "TechCorp"
}
```

এটা machine-friendly।

Use করবে যখন:

- code integration
- API building
- data extraction
- automation
- consistent schema দরকার

এখন exercise-এর code part explain করি।

তোমার file: `prompt_engineering.py`

ওখানে একটা class আছে: `PromptEngineer`

এর কাজ: different prompting techniques test করা।

Step 1: `__init__`

এখানে client আর model store করবে।

```
python
def __init__(self, api_key: str, model: str = "gpt-3.5-turbo"):
    self.client = OpenAI(api_key=api_key)
    self.model = model
```

মানে:

- API key দিয়ে OpenAI client বানালো
 - কোন model use করবে সেটা save করলো
-

Step 2: `zero_shot_prompt()`

```
python
def zero_shot_prompt(self, task: str) -> str:
    response = self.client.chat.completions.create(
        model=self.model,
        messages=[{"role": "user", "content": task}],
        temperature=0
    )
    return response.choices[0].message.content
```

এখানে:

- শুধু task পাঠানো হচ্ছে
- no examples
- no reasoning instruction
- no format instruction

temperature=0 কেন?

- যাতে output stable থাকে
- prompting technique compare করা easier হয়

Step 3: few_shot_prompt()

এখানে prompt manually build করা হচ্ছে।

```
python
```

```
prompt = "Here are some examples:\n\n"
```

তারপর loop:

```
python
```

```
for i, example in enumerate(examples, 1):
```

```
    prompt += f"Example {i}:\n"
```

```
    prompt += f"Input: {example['input']}\n"
```

```
    prompt += f"Output: {example['output']}\n\n"
```

তারপর actual task add:

```
python
```

```
prompt += f"Now, complete this task:\n{task}"
```

মানে final prompt looks like:

```
text
```

Here are some examples:

Example 1:

Input: I love this

Output: Positive

Example 2:

Input: I hate this

Output: Negative

Now, complete this task:

Input: It was okay

Output:

এতে model pattern follow করে।

Step 4: chain_of_thought_prompt()

```
python
def chain_of_thought_prompt(self, problem: str) -> str:
    cot_prompt = f"{problem}\n\nLet's think step by step and show the reasoning:"

    response = self.client.chat.completions.create(
        model=self.model,
        messages=[{"role": "user", "content": cot_prompt}],
        temperature=0
    )

    return response.choices[0].message.content
```

এখানে মূল trick একটাই:

```
python
"Let's think step by step and show the reasoning:"
```

এই line model-কে reasoning mode-এ ঠেলে দেয়।

Step 5: `structured_output_prompt()`

এখানে format অনুযায়ী instruction map করা আছে:

```
python
format_instructions = {
    "JSON": "Respond in valid JSON format. Use proper JSON syntax with quoted keys.",
    "markdown table": "Respond as a markdown table.",
    "YAML": "Respond in valid YAML format.",
    "CSV": "Respond in CSV format with headers."
}
```

তারপর:

```
python
instruction = format_instructions.get(output_format, f"Respond in {output_format} format.")

prompt = f"{task}\n\n{instruction}"
```

মানে task + format instruction combine হচ্ছে।

Step 6: `compare_approaches()`

এখানে 3টা technique compare করা হচ্ছে:

```
python
results['zero_shot'] = self.zero_shot_prompt(problem)
```

যদি examples থাকে:

```
python
results['few_shot'] = self.few_shot_prompt(problem, examples)
```

আর:

```
python
```

```
results['chain_of_thought'] = self.chain_of_thought_prompt(problem)
```

মান্নে same problem-এর জন্য 3 style output compare করবে।

এখন experiments explain করি।

Experiment 1: Zero-shot vs Few-shot

Goal: **example** দিলে **format consistency** বাড়ে কি না

ধরো task sentiment classification।

Zero-shot output:

```
text
```

```
The sentiment is neutral.
```

Few-shot output:

```
text
```

```
Neutral
```

Few-shot better কেন?

- concise
- same format
- parse করা easy
- explanation কম

Lesson: **Few-shot = format control**

Experiment 2: Chain-of-Thought

Goal: **reasoning prompt accuracy/clarity** বাড়ায় কি না

Without CoT:

```
text
```

```
$36
```

With CoT:

text

Step 1...

Step 2...

Answer: \$36

Benefit:

- answer verify করা যায়
- logic দেখা যায়
- subtle mistake ধরা যায়

Lesson: **CoT = better reasoning + transparency**

Experiment 3: Structured Output

Goal: **format specify** করলে **output machine-friendly** হয় কি না

Without format:

text

Name: John Smith, Age: 35...

With JSON:

json

```
{  
  
  "name": "John Smith",  
  
  "age": 35
```

Lesson: **data extraction task-এ always format specify** করো

Experiment 4: Comprehensive Comparison

এখানে একটা average speed problem দিয়েছে।

Important thing:

- zero-shot subtle detail miss করতে পারে
- few-shot format improve করতে পারে
- CoT hidden step ধরতে সাহায্য করে

Problem-এ stop time ছিল। CoT step-by-step করলে stop time include করা easier হয়।

Lesson: **complex word problem-এ CoT best**

Experiment 5: Real-world Application

এখানে multiple techniques combine করা হয়েছে:

- few-shot
- structured output

Task: product review থেকে structured data extract করা

এটা real-world useful কারণ:

- শুধু answer না
- parseable JSON দরকার
- pros/cons list দরকার
- rating দরকার

Lesson: বাস্তবে এক **technique** না, **combination use** হয়

এখন খুব important একটা comparison table দিচ্ছি:

Technique	কী করে	কখন use করবে	Weakness
Zero-shot	শুধু task দেয়	simple/common task	format inconsistent
Few-shot	examples দিয়ে pattern শেখায়	custom format/category	extra tokens লাগে

CoT	reasoning steps চায়	math/logic/multi-step	simple task-এ unnecessary
Structured output	exact format চায়	JSON/CSV/table extraction	format না মানলে retry লাগতে পারে

এখন “কখন কোনটা” খুব short rules:

- যদি task simple হয় → **Zero-shot**
- যদি exact label/format দরকার হয় → **Few-shot**
- যদি reasoning দরকার হয় → **CoT**
- যদি code-এ parse করতে হয় → **Structured output**
- যদি real-world extraction হয় → **Few-shot + Structured output**
- যদি complex reasoning + strict format হয় → **CoT + Structured output**

তোমার exam/revision-এর জন্য 5টা golden rule:

1. **Simple task** হলে **zero-shot** যথেষ্ট
2. **Consistent output** চাইলে **few-shot**
3. **Complex reasoning** চাইলে **CoT**
4. **Machine-readable output** চাইলে **JSON/CSV/table specify** করো
5. **Best results often come from combining techniques**

আরেকটা super simple example:

ধরো task:

text

Extract name and age from: Rahim, 25

- zero-shot:

text

The person's name is Rahim and age is 25.

- few-shot:

text

Name: Rahim

Age: 25

- structured output:

json

```
{"name": "Rahim", "age": 25}
```

- CoT:

text

Step 1: Identify the name = Rahim

Step 2: Identify the age = 25

Answer: Rahim, 25

দেখতেই পাচ্ছো, task অনুযায়ী best technique আলাদা।

এই lesson-এর main idea হলো: **LLM** থেকে ভালো **output** পেতে হলে **prompt**-টা **refine** করতে হয়। মানে, শুধু কিছু লিখে পাঠালেই হবে না — কী role, কী task, কী context, কী format, কী example দিচ্ছো — এগুলো carefully ঠিক করলে output অনেক better হয়।

সহজভাবে prompt-এর ৫টা অংশ:

- **Role** → AI কেমন persona নেবে
উদাহরণ: “Act as a pirate” / “Act as a marketing manager”
- **Task** → AI-কে আসলে কী করতে বলছো
উদাহরণ: “Recommend a local dish”
- **Context** → কাজের জন্য দরকারি background info
উদাহরণ: product details, ship name, target audience
- **Output format** → output কেমন হবে
উদাহরণ: paragraph, bullet points, JSON
- **Examples** → sample input-output দেখানো
এতে model desired pattern ধরতে পারে

lesson-এর সবচেয়ে important point: একবারে সব **change** না করে, একেকবারে একেক **component change** করে **output compare** করতে হবে।

এটাকেই systematic prompt refinement বলে। এতে বোঝা যায় কোন change-এর কারণে output improve হলো।

উদাহরণ:

- **Role** বদলালে tone/style বদলায়
- **Task** বদলালে কাজটাই বদলায়
- **Format** বদলালে structure বদলায়
- **Examples** বদলালে style/pattern influence হয়
- **Context** বদলালে factual content বদলায়

Best practices:

- clear and detailed instruction দাও
- role use করো
- complex task ছোট ছোট step-এ ভাঙো
- output format clearly বলো
- few-shot examples দাও
- relevant context দাও
- agent হলে tool description clear রাখো

Common pitfalls:

- **Ambiguity** → prompt vague
- **Insufficient context** → দরকারি info দাওনি
- **Too much/conflicting context** → model confused
- **Poor tool descriptions**
- **Expecting magical understanding** → model তোমার mind read করতে পারে না
- **Bias / hallucination**
- **Prompt injection / adversarial risk**

Marketing example-এ দেখিয়েছে:

- প্রথম prompt generic output দেয়
- context add করলে better হয়
- role + constraint + CTA add করলে output অনেক strong হয়

এক লাইনের takeaway: **Good prompt = clear task + right role + enough context + explicit format + useful examples + iterative testing**

অবশ্যই — এটা আসলে পুরো lessonটার core idea: **prompt** লম্বা করলেই ভালো হয় না, **prompt**-কে সঠিকভাবে **refine** করতে হয়।

সহজভাবে বললে, এই lesson তোমাকে শিখাচ্ছে:

- vague prompt দিলে model মোটামুটি answer দেয়
- refined prompt দিলে model **consistent, explainable, structured** answer দেয়
- refinement মানে random extra words না, বরং prompt-এর **Role + Task + Context + Rules + Output Format + Examples** improve করা

তোমার দেওয়া পুরো contentটা আমি Bangla-তে detail-এ ভেঙে বুঝাই।

প্রথমে main idea:

ধরো তুমি AI-কে বললে:

```
python
```

```
"Please categorize this email"
```

এতে AI একটা human-like sentence দেবে:

```
json
```

```
"This email appears to be a billing issue."
```

এটা ভুল না, কিন্তু automation-এর জন্য খুব useful না। কারণ program এটা parse করে সহজে কাজ করতে পারবে না।

কিন্তু যদি তুমি prompt refine করে বলো:

- তুমি কে? → role
- কী কাজ করবে? → task
- category মানে কী? → context
- output কেমন হবে? → format
- uncertainty হলে কী করবে? → rules

তাহলে output হবে এরকম:

```
json
```

```
{  
  
  "category": "Billing",  
  
  "summary": "Customer was charged twice...",  
  
  "urgency": "High",  
  
  "customer_id": "8675309"  
}
```

এখন এটা machine-readable, consistent, downstream system-friendly।

এটাই prompt instruction refinement-এর power।

এখন recipe exercise-এ এই same idea apply করা হয়েছে।

তোমার task ছিল: একটা recipe দেওয়া হবে, আর কিছু dietary restriction দেওয়া হবে:

- vegetarian
- vegan
- gluten-free
- dairy-free
- nut-free
- egg-free
- low sodium
- keto
- paleo
- kosher

LLM-কে বলতে হবে recipe এগুলো satisfy করে কি না।

কিন্তু simple prompt দিলে problem হয়।

১) Initial prompt-এ কী ছিল?

শুরুতে prompt ছিল roughly এইরকম:

```
python
```

```
Analyze the following recipe and determine whether it satisfies each dietary restriction.
```

```
Classify each as satisfied, not satisfied, or undeterminable.
```

```
Provide the response in JSON format.
```

এর সাথে recipe ingredients, instructions, আর restriction list inject করা হচ্ছিল Jinja2 template দিয়ে।

মানে model-কে শুধু broadly বলা হচ্ছিল:

- recipe দেখো
- restriction check করো
- JSON দাও

২) Initial prompt-এর সমস্যা কী?

Surface level-এ output ঠিকঠাক লাগতে পারে। যেমন spaghetti bolognese-এর জন্য:

- vegetarian → not satisfied
- vegan → not satisfied
- gluten-free → not satisfied
- dairy-free → not satisfied

এগুলো ঠিকই।

কিন্তু সমস্যা হলো:

- **explanation** নেই
- কোন **ingredient problem** করছে সেটা বলে না
- **ambiguity handle** করার **rule** নেই
- **undeterminable** কেন হলো সেটা **explain** করে না
- **different runs**-এ **inconsistent** হতে পারে
- “**kosher**” এর মতো **nuanced restriction**-এ **model guess** করতে পারে

অর্থাৎ answer আছে, কিন্তু **trustworthy reasoning** নেই।

এই lesson-এর একটা খুব important point: **Correct-looking answer ≠ reliable answer**

৩) Prompt component analysis কী?

Lesson-এ prompt-কে parts-এ ভেঙে দেখা হয়েছে। সাধারণত prompt-এর major components:

- **Role**
- **Task**
- **Context**
- **Output Format**
- **Examples**

এগুলো refine করলে output improve হয়।

চলো এক এক করে দেখি।

৪) Role কী?

Role মানে model-কে একটা identity দেওয়া।

যেমন:

```
python
```

```
You are a dietary consultant.
```


অথবা

```
python
```

```
You are an expert analyst for dietary restriction compliance.
```

এতে model-এর behavior more specialized হয়।

আগের vague prompt-এ role ছিল না। Refined prompt-এ role add করা হয়েছে।

কেন useful? কারণ model তখন generic assistant mode থেকে বের হয়ে domain-specific ভাবে respond করতে চেষ্টা করে।

৫) Task refinement কী?

শুধু “classify করো” বললে model minimal answer দিতে পারে।

কিন্তু refined task-এ বলা হয়:

- classify করো
- explanation দাও
- critical ingredients identify করো
- certainty না থাকলে undeterminable দাও
- guess করো না

এতে task clearer হয়।

মানে model বুঝে: “আমাকে শুধু label দিতে হবে না, justification-ও দিতে হবে।”

৬) Context refinement কী?

এটা সবচেয়ে important অংশগুলোর একটা।

ধরো তুমি শুধু বললে:

```
python
```

```
Check if recipe is vegan.
```

Model vegan-এর definition জানে, কিন্তু prompt-এ explicitly দিলে consistency বাড়ে।

যেমন:

- Vegetarian: no meat, poultry, fish
- Vegan: no animal-derived ingredients
- Gluten-free: no wheat/barley/rye
- Dairy-free: no milk, butter, cheese, cream
- Nut-free: no tree nuts or peanuts
- Egg-free: no eggs
- Kosher: depends on specific preparation and ingredient sourcing, so often undeterminable unless clearly stated

এখন model-এর কাছে decision rules clearer।

এতে benefit:

- less assumption
- more reproducible output
- edge case-এ better handling

৭) Output format refinement কী?

Initial prompt-এ শুধু “JSON format” বলা হয়েছিল।

কিন্তু refined prompt-এ বলা হলো, প্রতিটা restriction-এর জন্য object থাকবে:

```

json
{
  "classification": "satisfied | not satisfied | undeterminable",
  "explanation": "short reason",
  "critical_ingredients": ["ingredient1", "ingredient2"]
}

```

এটা huge improvement।

কারণ এখন:

- output consistent
- parse করা easy
- explanation structured
- downstream app use করতে পারবে

এটাই structured prompting-এর practical value।

৮) Iteration 1-এ কী change করা হলো?

প্রথম refinement-এ মূলত ৩টা বড় improvement:

- dietary restriction-এর **definitions** add করা
- classification-এর **rules** add করা
- output-এর **detailed structure** add করা

Rules-এর idea ছিল এরকম:

- **Satisfied** only if certain
- **Not satisfied** if any ingredient clearly violates
- **Undeterminable** if info insufficient
- প্রতিটিয় explanation দিতে হবে
- critical ingredients identify করতে হবে

এটা খুব important logic।

কারণ model-কে বলা হচ্ছে: **certainty** ছাড়া **satisfied** বলবে না

এতে false positive কমে।

৯) Spaghetti example-এ Iteration 1 কেন better?

আগে model শুধু বলত:

```
json
```

```
"vegetarian": "not satisfied"
```

এখন বলে:

```
json
```

```
"vegetarian": {
```

```
  "classification": "not satisfied",
```

```
  "explanation": "Contains ground beef, which is meat and violates vegetarian restrictions.",
```

```
  "critical_ingredients": ["ground beef"]
```

```
}
```

এখন answer:

- transparent
- auditable
- easier to debug

যদি model ভুলও করে, তুমি বুঝতে পারবে কোথায় ভুল **reasoning** করেছে।

এটা prompt refinement-এর একটা বড় লাভ: **wrong answer** হলেও **wrong reasoning visible** হয়

১০) Iteration 2-এ কী add করা হলো?

এখানে focus ছিল **ambiguity handling** + **example/few-shot guidance**।

এই অংশ lesson-এর খুব important।

কারণ real-world recipe-তে অনেক ingredient ambiguous হয়:

- broth
- stock
- oil
- sauce
- optional nuts
- cheese type
- flour type
- soy sauce
- gelatin
- shortening

যদি prompt-এ ambiguity rules না দাও, model random assumption করতে পারে।

তাই prompt-এ rules add করা হয়, যেমন:

- vegetable oil → plant-based ধরে নাও unless otherwise noted
- unspecified broth/stock → recipe-এর main protein-এর সাথে match করতে পারে
- optional ingredient restriction violate করলে certainty কমে যেতে পারে
- unclear ingredient হলে undeterminable consider করো

এতে model-এর decision policy clearer হয়।

১১) Few-shot example কেন add করা হলো?

Prompt-এ একটা example analysis add করা হয়েছিল, যেমন “Basic Pancakes”।

Few-shot example model-কে দেখায়:

- expected reasoning style কী
- output structure exactly কেমন
- ambiguity হলে কীভাবে explain করতে হবে

এটা শুধু instruction-এর চেয়ে stronger signal দেয়।

কারণ model শুধু rule পড়ে না, **pattern imitate**-ও করে।

তাই examples দিলে:

- format consistency বাড়ে
 - explanation quality improve হয়
 - edge case handling better হয়
-

১২) Vegetable Stir Fry example-এ কী শেখানো হলো?

Lesson-এ দেখানো হয়েছে refined prompt দিয়েও model perfect না।

Vegetable stir fry-তে model vegetarian ঠিক ধরেছে। কিন্তু vegan নিয়ে questionable reasoning দিয়েছে:

- soy sauce animal-derived additive থাকতে পারে
- tofu animal-derived বলেছে, যা ভুল

এখানে lesson-এর important takeaway: **prompt refinement improves output, but does not guarantee correctness**

মানে:

- prompt better হলে reasoning better হয়
- transparency বাড়ে
- ambiguity handling improve হয়
- কিন্তু model এখনও factual mistake করতে পারে

এটা খুব realistic lesson।

১৩) Chocolate Chip Cookies example-এ কী interesting?

এখানে “chopped nuts (optional)” ছিল।

Nut-free classification-এ model “undeterminable” বলেছে, কারণ nuts optional।

এটা actually nuanced reasoning।

কারণ:

- যদি nuts add করা হয় → nut-free না
- যদি না add করা হয় → nut-free হতে পারে

তাই certainty না থাকলে “undeterminable” reasonable।

এখানে refined prompt model-কে binary yes/no থেকে বের করে **careful classification** করতে সাহায্য করেছে।

১৪) “Satisfied / Not satisfied / Undeterminable” logic কেন important?

এই 3-way classification খুব useful।

- **Satisfied** → recipe clearly restriction meet করে
- **Not satisfied** → clear violation আছে
- **Undeterminable** → info enough না

অনেক student mistake করে সবকিছু yes/no ভাবে। কিন্তু real-world data messy।

LLM-কে যদি তুমি শুধু yes/no force করো, model guess করবে। আর যদি “undeterminable” option দাও, model uncertainty express করতে পারে।

এটা safer prompting।

১৫) এই exercise-এর আসল learning কী?

এই exercise recipe analysis নিয়ে হলেও আসল skill recipe-specific না।

আসল skill হলো: **How to iteratively improve a prompt when the first answer is not good enough**

Processটা roughly এমন:

1. initial prompt লেখো
2. output দেখো
3. shortcomings identify করো
4. missing context add করো
5. task clarify করো
6. output format tighten করো
7. ambiguity rules add করো
8. example add করো
9. আবার test করো

10. compare করো

এটাই iterative prompt development।

১৬) “Longer prompt” vs “Better prompt”

Lesson-এর শুরুতে এই question ছিল: লম্বা prompt কি সবসময় better?

Answer: না।

কারণ bad long prompt হতে পারে:

- verbose
- repetitive
- unclear
- conflicting
- unfocused

আর good refined prompt হয়:

- specific
- structured
- constrained
- contextualized
- testable

মানে: **More words $\neq$ more quality Better instructions = better output**

১৭) এই lesson-এর prompt engineering principles কী কী?

এই lesson থেকে তুমি practically শিখলে:

- role assign করো
- task explicit করো
- domain definitions দাও
- decision rules দাও
- uncertainty handling define করো
- structured output চাহো
- examples দাও
- test on multiple inputs
- compare outputs
- refine iteratively

এগুলো শুধু recipe task-এ না, almost সব LLM task-এ use করা যায়।

১৮) Real-world use cases কোথায়?

এই same refinement pattern use করা যায়:

- customer support ticket classification
- resume screening
- sentiment analysis
- medical note extraction
- legal clause review
- product review moderation
- document summarization
- RAG answer formatting
- agent tool output structuring

যেখানে raw natural language answer enough না, structured reliable output দরকার — সেখানে এই approach খুব useful।

১৯) কেন explanation + critical ingredients এত valuable?

কারণ শুধু label দিলে তুমি জানো না model কী basis-এ decision নিল।

কিন্তু explanation দিলে:

- human review easy
- debugging easy
- trust বাড়ে
- wrong assumptions ধরা যায়

আর critical ingredients দিলে:

- exact trigger বোঝা যায়
- downstream UI-তে highlight করা যায়
- audit trail পাওয়া যায়

যেমন:

```
json
```

```
"critical_ingredients": ["ground beef", "parmesan cheese"]
```

এটা plain “not satisfied” থেকে অনেক বেশি useful।

২০) Kosher-এর মতো category কেন tricky?

কারণ কিছু restriction ingredient list দেখে fully determine করা যায় না।

Kosher depend করতে পারে:

- ingredient source
- preparation method
- meat-dairy mixing
- certification
- kitchen handling

তাই refined prompt-এ model-কে শেখানো হয়: যদি enough info না থাকে, confidently satisfied বলো না — **undeterminable** বলো।

এটা responsible prompting।

২১) Jinja2 এখানে কেন use করা হয়েছে?

Notebook-এ `format_prompt` function Jinja2 দিয়ে prompt template fill করছিল।

এটার কাজ:

- recipe ingredients bullet list বানানো
- instructions numbered list বানানো
- restrictions list inject করা
- একই prompt structure multiple recipe-তে reuse করা

মানে prompt engineering শুধু wording না, **prompt templating**-ও important।

Production system-এ এটা খুব common।

২২) এই lesson-এর final comparison কী?

Initial prompt:

- role নেই
- context কম
- task vague
- output loosely defined
- example নেই

- ambiguity handling নেই

Final refined prompt:

- role আছে
- dietary definitions আছে
- classification rules আছে
- explanation required
- critical ingredients required
- ambiguity handling আছে
- example আছে
- structured JSON আছে

ফলে output:

- more transparent
- more consistent
- more useful
- more debuggable

২৩) একটা compact mental model দিচ্ছি

যখন prompt ভালো result দিচ্ছে না, তখন এই checklist use করো:

- **Role:** model কে?
- **Task:** exactly কী করবে?
- **Context:** decision নেওয়ার জন্য কী info দরকার?
- **Rules:** ambiguity/uncertainty কীভাবে handle করবে?
- **Format:** output exact কেমন হবে?
- **Examples:** desired pattern দেখানো হয়েছে?
- **Evaluation:** multiple test case-এ check করা হয়েছে?

এটাই prompt refinement framework।

২৪) এই lesson-এর one-line essence

Prompt refinement মানে **prompt**-এ এমন **structure, rules, context**, আর **examples** যোগ করা যাতে **model** শুধু **answer** না দেয়, বরং **useful, consistent, explainable answer** দেয়।

চাইলে আমি এখন তোমার জন্য এটা আরও ২ভাবে দিতে পারি:

1. একদম **short revision notes**
2. **zero-shot vs refined prompt vs few-shot comparison table**

যেটা চাও, আমি Bangla-তেই করে দিচ্ছি।

“Critical ingredients” বলতে prompt-এর সেই সবচেয়ে জরুরি অংশগুলো বোঝায়, যেগুলো ভালো output পাওয়ার জন্য দরকার। এই course context অনুযায়ী main ingredients হলো:

- **Role** — model কোন ভূমিকায় কাজ করবে
- **Task** — ঠিক কী করতে হবে
- **Context** — কাজের জন্য দরকারি background/info
- **Output format** — answer কী format-এ দেবে
- **Examples** — sample input/output, especially few-shot এ

এক লাইনে:

```
text
```

```
Good prompt = Role + Task + Context + Format + Examples
```

ছোট example:

```
text
```

You are a marketing expert.

Write a product description for a reusable coffee cup.

Target audience: eco-conscious students.

Return 3 bullet points and 1 CTA.

```
Example style: short, friendly, persuasive.
```

অবশ্যই — এটা course-এর পুরো lesson-এর **detailed but simple** explanation।

এই lesson-এর main idea হলো: **LLM**-কে নতুন **task** করতে সবসময় **retrain** বা **fine-tune** করতে হয় না। অনেক সময় শুধু prompt দিয়েই কাজ করানো যায় — examples ছাড়া হলে **zero-shot**, আর কয়েকটা example দিলে **few-shot**।

- **Zero-shot learning:** model-কে কোনো example না দিয়ে শুধু instruction দাও।
 - text
- Write a product description for this new silverware set.
এখানে model তার pretraining-এর আগের knowledge use করে কাজটা করার চেষ্টা করে।
- **Few-shot learning:** task-এর 1–fewটা example prompt-এ দাও, যাতে model pattern শিখে নেয়।
 - text

Example 1: Product A description ...

Example 2: Product B description ...

- Now write one for Product C.

এটা especially useful যখন তুমি **specific tone, format, style, or behavior** চাও।

আরেকটা important term ছিল **inference time**।

এটা হলো model যখন নতুন input পায়, তখন prediction/output দিতে যে সময় নেয়। এটা **training time** থেকে আলাদা।

Training time-এ model শেখে; inference time-এ model learned knowledge use করে answer দেয়।

এই lesson-এ marketing example দিয়ে দেখিয়েছে:

- নতুন product-এর জন্য description লিখতে হলে
 - যদি শুধু specs দাও → zero-shot
 - যদি আগের brand-style descriptions-ও দাও → few-shot
- Few-shot সাধারণত বেশি consistent output দেয়, কারণ model বুঝতে পারে company tone কেমন।

Ruby example-টা খুব important:

- ChatGPT পুরনো knowledge নিয়ে trained ছিল
- Ruby 3.2-এর নতুন `Data.define` method সে naturally জানত না
- zero-shot prompt দিয়েও ঠিক output দেয়নি
- কিন্তু few-shot-এ documentation example দেওয়ার পর ঠিক method use করেছে

এখান থেকে takeaway:

- zero-shot ভালো যখন task simple বা model already enough জানে
- few-shot ভালো যখন task **new, niche, updated, or format-sensitive**

Robot example-এ দেখিয়েছে few-shot শুধু text generation-এর জন্য না — **action generation**-এর জন্যও useful।

Human instruction → ChatGPT → robot action sequence।

মানে, few-shot examples দিয়ে model-কে শেখানো যায় natural language থেকে executable steps বানাতে।

তারপর limitations:

- **Accuracy:** answer plausible হতে পারে, কিন্তু সবসময় correct না
- **Data quality:** few-shot examples খারাপ হলে output-ও খারাপ হবে
- **Transferability:** prompt-এর context permanent learning না; context reset হলে model “ভুলে যায়”

মানে:

```
text
```

```
Prompting != permanent training
```

Future improvements section-এ 3টা জিনিস বলেছে:

- **Bigger LLMs** → better generalization
- **RAG** → external docs/database এনে prompt-এ যোগ করা
- **Multimodal AI** → শুধু text না, image + text + other inputs use করা

Quiz answer-গুলোও likely এগুলো:

1. examples ছাড়া prior knowledge use করে task করা → **Zero-shot learning**
2. fruit category শেখাতে few examples দেওয়া → **Few-shot learning**
3. new data-তে prediction দিতে যে সময় লাগে → **Inference time**
4. robot actions guide করতে critical item → **well-chosen examples/prompts/instructions**
সবচেয়ে safe answer: **few-shot examples in the prompt**

সব মিলিয়ে lesson-এর core message:

- **Zero-shot = no examples**
- **Few-shot = a few examples**
- দুটোই **inference time techniques**
- few-shot usually বেশি reliable when task needs format/style/domain guidance
- কিন্তু এগুলো fine-tuning-এর replacement না সব ক্ষেত্রে

চাইলে আমি এখন এটা আরও easy করে **Bangla short notes, quiz answers only**, বা **zero-shot vs few-shot comparison table** বানিয়ে দিতে পারি।

অবশ্যই — **inference-time techniques** মানে হলো এমন কৌশল, যেগুলো model-এর **training** শেষ হওয়ার পরে, নতুন input এ response generate করার সময় ব্যবহার করা হয়।

সহজভাবে:

- **Training time** = model শেখে
- **Inference time** = model যা শিখেছে, সেটা ব্যবহার করে answer দেয়

তাই **zero-shot** আর **few-shot** inference-time techniques, কারণ:

- আমরা model-এর weights change করি না
- model-কে retrain করি না
- শুধু prompt/input বদলাই, যাতে output better হয়

একটু breakdown করি:

- **Zero-shot**
 - শুধু task বলি
 - example দিই না
 - যেমন:

text

```
Classify this email as billing, technical, or general support.
```

- **Few-shot**

- task-এর সাথে 2–5টা example দিই
- model pattern ধরে better output দেয়
- যেমন:

```
text
```

```
Email: I was charged twice.
```

```
Category: billing
```

```
Email: The app crashes on login.
```

```
Category: technical
```

```
Now classify:
```

```
Email: I can't reset my password.
```

```
Category:
```

কেন এগুলো inference-time?

- কারণ model-এর ভিতরের knowledge permanently update হচ্ছে না
- ওই session/prompt-এর context ব্যবহার করে answer দিচ্ছে
- prompt সরিয়ে দিলে সেই “learning” আর থাকে না

এগুলো আর কী কী inference-time technique হতে পারে?

- **Prompt engineering** — instruction clear করা
- **Role prompting** — “You are a helpful tax assistant”
- **Output formatting constraints** — “Return JSON only”
- **Few-shot examples** — pattern দেখানো
- **Context injection** — relevant info prompt-এ দেওয়া
- **RAG** — retrieval করে fresh info এনে prompt-এ যোগ করা
- **Chain-of-thought style guidance** / step-by-step prompting

কেন useful?

- retraining ছাড়াই দ্রুত improve করা যায়
- niche task handle করা যায়
- updated info/context দেওয়া যায়
- format consistency বাড়ে

Limitations:

- এটা **real learning** না
- bad prompt হলে bad output
- context window-এর limit আছে
- prompt-এর বাইরে model knowledge update হয় না
- few-shot example ভুল হলে output-ও ভুল হতে পারে

এক লাইনে:

- **Inference-time techniques** = model retrain না করে, response generate করার সময় prompt/context/examples বদলে performance improve করার উপায়।

চাইলে আমি এটা next message-এ একটা **Bangla comparison table** এ দেখাতে পারি: **training-time vs inference-time + zero-shot vs few-shot**.

- Prompt engineering = better prompt, better output
- Zero-shot = no examples
- Few-shot = a few examples
- Inference time = new input এ output generate করার সময়
- Refine prompts one change at a time
- Output format helps structure
- Few-shot helps tone/format/consistency
- Prompting is temporary, not permanent learning
- **Undeterminable** avoids forced wrong answers
- Better prompts improve reliability, not certainty